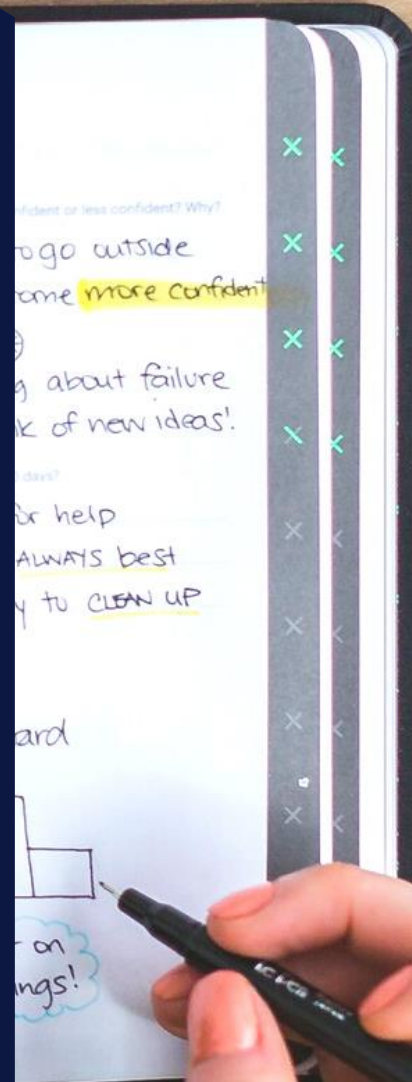




Tarea # 1

Unidad 3. Programación
del lado del cliente



Actividades

Actividad: Desarrollo de una aplicación web para el Catálogo de Productos SPA + API propia

Objetivo General

Desarrollar una aplicación web de una sola página (SPA) con Vue 3 (Composition API, SFC) que consume un API REST creado por el propio estudiante (Unidad 2). La solución debe demostrar reactividad, componentes, comunicación por props/eventos, composables, manejo de rutas, carga asíncrona de datos, control de errores y lazy loading de vistas.

Requisitos Previos

- Mantener la organización de los archivos una estructura clara
- El código resultante debe ser:
 - Entregado comprimido en un archivo .zip
 - Subido a un repositorio público en GitHub con instrucciones mínimas de uso (README.md)

Descripción general

El estudiante desarrollará una aplicación web completa llamada “MercApp”: catálogo de productos con fichas detalladas, buscador, filtro por categoría y un carrito simple (en memoria). El API expone productos y categorías (mínimo), con operaciones de lectura y creación/edición/borrado de productos.

Tareas Obligatorias

No.	Tarea
1	Diseño del esquema del API y modelos de datos • GET /api/products, GET /api/products/:id, POST /api/products, PUT/PATCH /api/products/:id, DELETE /api/products/:id • GET /api/categories • Modelo Product (id, name, description, price, imageUrl, categoryId, stock), Category (id, name).

2	Implementación del API REST con validación mínima (campos obligatorios, tipos), manejo de errores (400/404/500).
3	Semilla de datos y persistencia. Crear un script para poblar 8–12 productos y 3–5 categorías.
4	Bootstrap del proyecto Vue 3 con Vite y estructura SFC. Crear proyecto, limpiar plantilla, configurar alias, y assets.
5	Routing (SPA): rutas principales y dinámica por id. Configurar Vue Router: • Rutas: / (Home/Catálogo), /product/:id (detalle), /cart, /about. • Ruta 404.
6	Listado y búsqueda con reactividad y propiedades computadas. Crear una vista Home que consume GET /api/products, con: • Buscador por nombre/descripción. • Filtro por categoría (usa GET /api/categories). • Propiedad computada para lista visible (filtrada).
7	Detalle de producto y comunicación por props/eventos Acciones: vista /product/:id que obtiene descripción completa; componente ProductCard reutilizable: Prop product (objeto). Evento personalizado added-to-cart desde botón “Añadir”.
8	Composables y manejo de peticiones. Crear useFetch o useApi (genérico) y useProducts (específico): • Estados: data, loading, error. • Reintento simple (1 vez) y cancelación opcional.
9	Formulario de alta/edición con v-model y validación para crear/editar producto (modal o vista /product/new y /product/:id/edit): • Campos: nombre, precio numérico > 0, categoría obligatoria, stock ≥ 0, URL de imagen válida.

10	Lazy loading de vistas y <Suspense>. Cargar perezosamente al menos 2 rutas (p. ej., /about, /cart) con defineAsyncComponent o import() en router y usar <Suspense> con fallback.
11	Manejo de estado del carrito. agregar/quitar items, cantidad por producto, total calculado (computed), persistencia en localStorage
12	Documentación y evidencias. Acciones: Crear un repositorio en GitHub Realizar al menos tres commits en fechas diferentes. Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso. Coloca la URL del repositorio de GitHub en un archivo readme.txt dentro del ZIP entregado.

INSTRUCCIONES PARA LA REALIZACIÓN DE LA ACTIVIDAD DE LA UNIDAD 3

1. Crear un repositorio en GitHub y trabajar de forma síncrona con el proyecto
2. Generar un archivo README.md indicando los datos del estudiante, las funcionalidades implementadas e instrucciones básicas de uso
3. Coloca la URL de su repositorio de GitHub en un archivo readme.txt dentro de la carpeta principal del proyecto
4. Una vez concluido su desarrollo, **comprimir la carpeta en un archivo ZIP y enviar al AVAC** con el siguiente formato “Actividad3-Apellido1-Nombre1.zip”.

Rúbrica:

Criterios	Bajo (0%)	Regular (30%)	Medio (60%)	Alto (100%)	Sub-puntaje máximo
1. Funcionalidad Ejecuta correctamente las tareas definidas	No cumple tareas o presenta fallos graves (errores que bloquean, vistas que no cargan, API no responde). 0	Cumple hasta 4 tareas de forma parcial o incompleta. Flujos interrumpidos o sin validaciones mínimas. 1.8	Cumple la mayoría de tareas (5-9), CRUD casi completo, navegación básica y consumo parcial del API. Estados de carga/vacío presentes pero inconsistentes.	Cumple todas las tareas (≥10), CRUD completo, filtros y búsquedas funcionales, rutas correctas, estados coherentes de carga/éxito/error, validaciones completas y manejo	6

			3.6	consistente de errores. 6	
2. Uso de tecnologías	No utiliza las tecnologías requeridas o las usa de forma incorrecta. 0	Uso limitado o incorrecto de tecnologías principales. Backend: API con endpoints inconsistentes o sin CORS. Frontend: Vue 3 con SFC mínimos, sin manejo de estado, peticiones improvisadas. 0.72	API REST funcional con Express (o similar) y persistencia básica. Vue 3 (Composition API) con consumo del API, vue-router básico y manejo de estado limitado. Validaciones y documentación incompletas. 1.44	API REST modular, validada y documentada; CORS, manejo de errores y persistencia. Vue 3 con SFCs bien estructurados, Pinia para estado global, vue-router con rutas dinámicas/guardias, consumo mediante servicios, formularios con validación completa y pruebas básicas. 2.4	2.4
3. Diseño visual y responsivo	No hay diseño visual; vistas rotas o sin estilos; sin diseño responsivo. 0	Vistas muy básicas, con estructura confusa; responsividad limitada; escasa reutilización de componentes. 0.28	Diseño funcional pero poco consistente. Responsividad aceptable. Uso parcial de componentes reutilizables y accesibilidad limitada. 0.96	Diseño atractivo y uniforme. Layout responsivo mobile-first. Componentes reutilizables con props/slots, estilos consistentes, accesibilidad básica (semántica, contraste, foco) y micro-interacciones suaves. 1.6	1.6
4. Organización y entrega del proyecto Estructura de carpetas, uso de GitHub, documentación	No utiliza GitHub. El código está desorganizado y sin comentarios. No entrega el proyecto en ZIP. 0	Estructura poco clara; uso mínimo de GitHub (pocos commits, sin mensajes descriptivos). README	Estructura aceptable. Repositorio funcional pero con historial de commits irregular. README con instrucciones	Proyecto organizado (backend/frontend separados o monorepo limpio), commits descriptivos, README completo (instalación, ejecución, rutas API, usuarios de	2

		escueto o desactualizado. 0.36	parciales y sin .env.example. 1.2	prueba), .env.example y notas de despliegue. 2	
Totales					12